BRACT's

Vishwakarma Institute of Technology, Pune

Practical Implementation Sheet

Department: CSE-AI Semester: 5 Academic Year: 2026-27

Division: C Batch: 2 Practical No: 3

Roll no: 44 PRN No: 12411367 Name of Student: **Sarthak Kokadwar**

Subject Name : Deep Learning

Problem Statement :

Implement forward propagation and backpropagation using TensorFlow/Keras. Analyze the effect of different learning rates and the number of epochs on model performance

Algorithm :

Forward Propagation and Backpropagation using TensorFlow/Keras

- Import the required Python libraries for TensorFlow, Keras, NumPy, Matplotlib, and dataset handling.
- Load the IMDB dataset from the TensorFlow/Keras datasets library.
- Separate the dataset into training and testing sets.
- Preprocess the text data by limiting the vocabulary size and padding all input sequences to a fixed length using `pad_sequences()`.
- Create a Sequential Neural Network model.
- Add an Embedding layer to convert words into dense vector representations.
- Add a Global Average Pooling layer to reduce the sequence dimensions.
- Add one or more Dense (hidden) layers with the ReLU activation function.
- Add an output Dense layer with the Sigmoid activation function for binary sentiment classification.
- Compile the model using the Binary Crossentropy loss function, Adam optimizer, and Accuracy as the evaluation metric.
- Define different learning rates (e.g., **0.1, 0.01, and 0.001**) for performance comparison.
- Train the model using the training dataset with the `fit()` method for a fixed number of epochs.

- During **forward propagation**, pass the input data through all network layers to generate predictions.
- Compute the loss by comparing the predicted outputs with the actual target labels.
- Perform **backpropagation** to calculate gradients of the loss with respect to the model parameters.
- Update the weights and biases using the optimizer and the selected learning rate.
- Repeat the training process using different learning rates and different numbers of epochs (e.g., **5, 10, and 15 epochs**).
- Evaluate the trained model using the testing dataset with the `evaluate()` method.
- Record the testing accuracy and loss for each learning rate and epoch combination.
- Plot the Training Accuracy and Validation Accuracy curves against the number of epochs.
- Plot the Training Loss and Validation Loss curves to visualize the learning process.
- Compare the performance of the model for different learning rates and epoch values.
- Analyze how the learning rate affects the convergence speed and model accuracy.
- Analyze how increasing the number of epochs affects the training loss, accuracy, and the possibility of overfitting.
- Identify the learning rate and epoch combination that provides the best overall model performance.
- Display the final results and draw conclusions based on the experimental observations.

Code :

1. Import Required Libraries, Load Dataset $ Create Model :

```
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
from tensorflow.keras.optimizers import Adam
import matplotlib.pyplot as plt
import pandas as pd
from tensorflow.keras.datasets import imdb

(X_train, y_train), (X_test, y_test) = imdb.load_data(num_words=10000)
x_train=x_train.reshape(-1,784)/255.0
x_test=x_test.reshape(-1,784)/255.0
def create_model(lr):
    m=Sequential([Dense(128,activation='relu',input_shape=(784,)),
                  Dense(64,activation='relu'),
                  Dense(10,activation='softmax')])
    m.compile(optimizer=Adam(learning_rate=lr),
              loss='sparse_categorical_crossentropy',
              metrics=['accuracy'])
    return m
```

2. Compare Different Learning Rates :

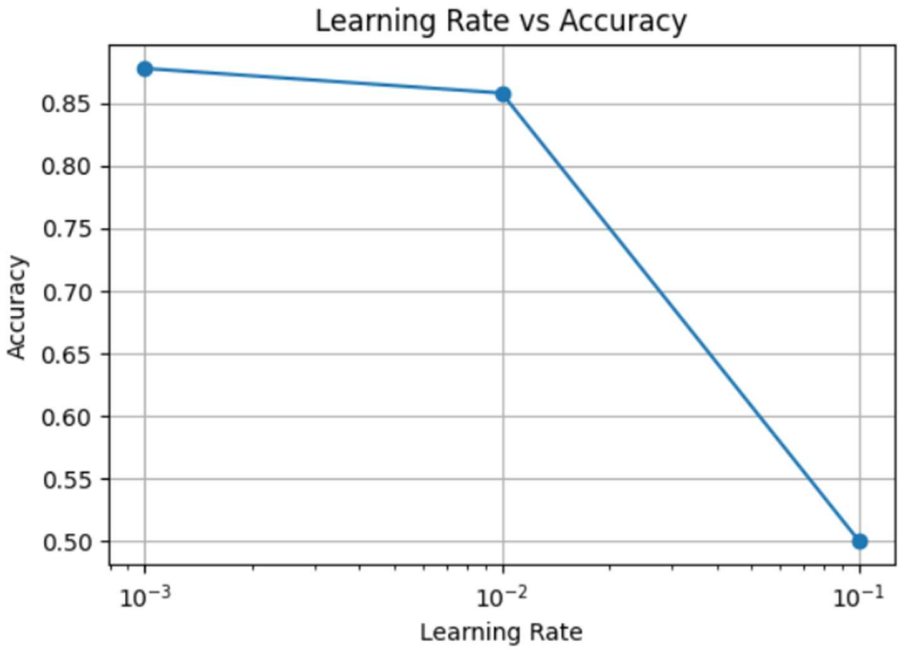
```
learning_rates=[0.1,0.01,0.001]
results=[]
for lr in learning_rates:
    model=create_model(lr)
    model.fit(x_train,y_train,epochs=5,batch_size=128,verbose=0)
    loss,acc=model.evaluate(x_test,y_test,verbose=0)
    results.append([lr,acc])
df=pd.DataFrame(results,columns=['Learning Rate','Accuracy'])
print(df)
plt.plot(df['Learning Rate'],df['Accuracy'],marker='o')
plt.xscale('log'); plt.grid(); plt.show()
```

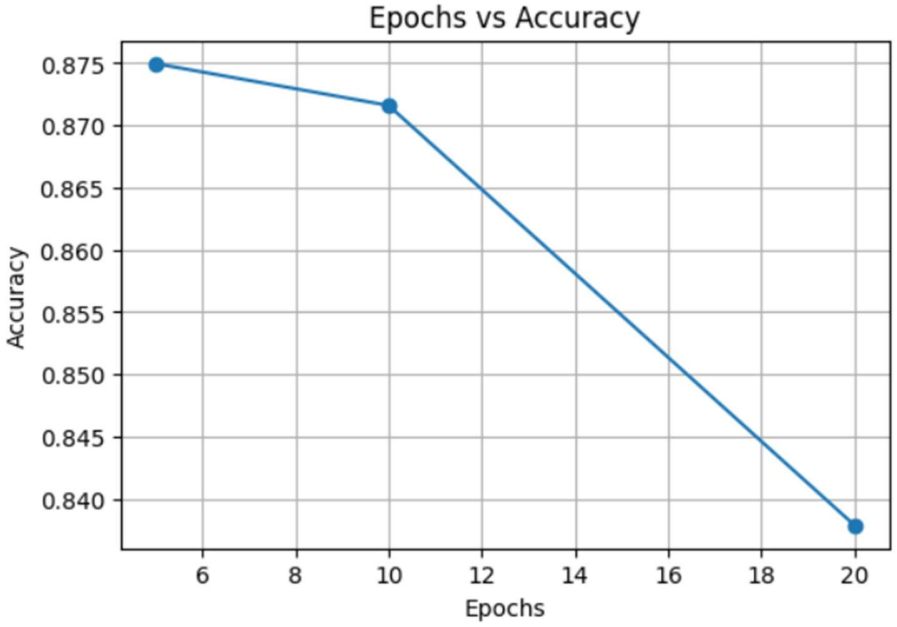
3. Compare Different Epochs, Train Final Model, Plot Graphs $ Evaluate :

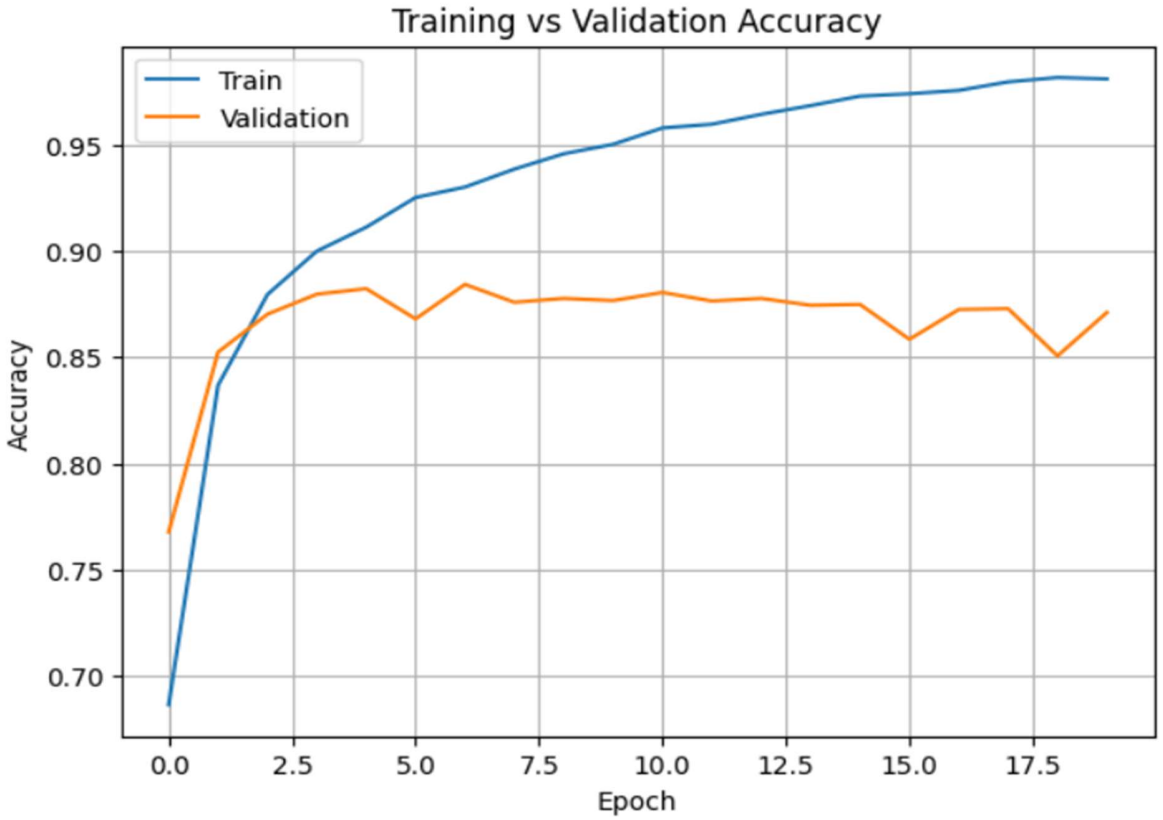
```
epochs_list=[5,10,20]
res=[]
for ep in epochs_list:
    model=create_model(0.001)
    h=model.fit(x_train,y_train,validation_split=0.2,epochs=ep,batch_size=128,ver
bose=0)
    loss,acc=model.evaluate(x_test,y_test,verbose=0)
    res.append([ep,acc])
edf=pd.DataFrame(res,columns=['Epochs','Accuracy'])
print(edf)
plt.plot(edf['Epochs'],edf['Accuracy'],marker='o')
plt.grid(); plt.show()

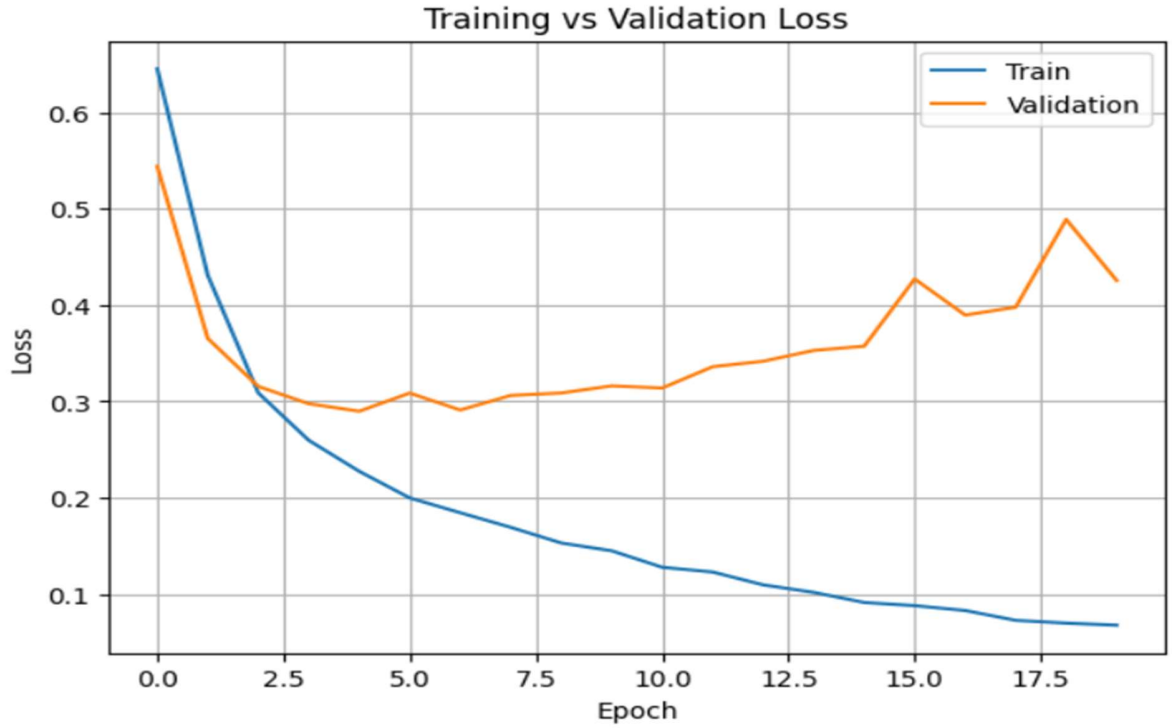
model=create_model(0.001)
history=model.fit(x_train,y_train,validation_split=0.2,epochs=20,batch_size=128)
plt.plot(history.history['accuracy']); plt.plot(history.history['val_accuracy']);
plt.legend(['Train','Val']); plt.show()
plt.plot(history.history['loss']); plt.plot(history.history['val_loss']);
plt.legend(['Train','Val']); plt.show()
print(model.evaluate(x_test,y_test))
```

OUTPUT :


Learning Rate vs Accuracy
Accuracy
0.85
0.80
0.75
0.70
0.65
0.60
0.55
0.50
10^{-3}
10^{-2}
10^{-1}
Learning Rate


Epochs vs Accuracy
Accuracy
0.875
0.870
0.865
0.860
0.855
0.850
0.845
0.840
6
8
10
12
14
16
18
20
Epochs


Training vs Validation Accuracy
Train
Validation
Accuracy
0.95
0.90
0.85
0.80
0.75
0.70
0.0
2.5
5.0
7.5
10.0
12.5
15.0
17.5
Epoch


Training vs Validation Loss
Train
Validation
Loss
0.6
0.5
0.4
0.3
0.2
0.1
0.0
2.5
5.0
7.5
10.0
12.5
15.0
17.5
Epoch

- **Conclusion:**

In this experiment, a neural network was implemented using TensorFlow/Keras to perform sentiment classification on the IMDB movie review dataset. The model successfully learned to classify reviews as positive or negative through forward propagation and backpropagation. The effect of different learning rates and numbers of epochs was analyzed, showing that the learning rate influences the speed and stability of convergence, while the number of epochs affects the model's learning and overall accuracy. The training and validation accuracy/loss graphs demonstrated the learning behavior of the model, and the best-performing combination of learning rate and epochs provided high classification accuracy with good generalization.